［問題13］

n（≦100）個の**倍精度実数**x_0, x_1, ・・・, x_{n-1}をキーボードから入力し、その合計，平均を求めるプログラムを関数を用いて作ります。例えば、プログラムは次のようになります。このプログラムは /home/pub/sample/prog13.c にあるので、続きを完成させなさい。

ただし、**2つの関数 inputData() と sumData() は、コメントで示された外部仕様通りに実装すること。**　　(prog13.c)

```
#include <stdio.h>
#include <hamakou.h>

#define MAX_SIZE 100

                                ;
                                 ;

main()
{
  double x[MAX_SIZE]; // データを格納する配列
  double s;           // 合計
  double h;           // 平均
  int    n;           // 入力したいデータの個数
  int  num;           // 実際に入力できたデータの個数

  n = getint("入力データの数：");
  // n個のデータを配列xへ入力
  if ((num = inputData( ,  )) <  ) {  // 入力できないデータがあった時は警告
    printf("\n入力データ数が配列の上限を超えました。\n");
    printf("入力できたデータのみについて集計します。\n");
  }
  s = sumData(   ,  );  // 配列xに格納されているデータの合計を求める
  h =        ;          // 平均を計算する
  printf("\n");

  printf("合計＝%g, 平均＝%g\n", s, h);

  return(0);
}
//-----------------------------------------------------------------------
//  実数データを配列xに入力し、実際に入力できたデータの個数を返す
//    第１引数 n   ：入力したいデータの個数
//    第２引数 x[]：倍精度実数データを格納する配列
//    戻 り 値     ：実際に入力したデータ数（但し、最大値は配列要素数の上限）
//-----------------------------------------------------------------------
    inputData(     ,           )
{
}

//-----------------------------------------------------------------------
//   配列xに格納されているn個の実数データの合計を求める
//    第１引数 n   ：配列に格納されているデータの個数
//    第２引数 x[]：倍精度実数データが格納されている配列
//    戻 り 値     ：配列xに格納されている倍精度実数データの合計
//-----------------------------------------------------------------------
       sumData(     ,           )
{
}
```

［問題14］

n行n列（1≦n≦50）の文字列が与えられたとき（この文字列をマップ、nをマップのサイズと呼ぶことにします）、以下の各問いに答えなさい。ただし、マップを表すデータは次のような形式で与えられるものとします。

（マップデータの形式）

n　　← マップのサイズ（以下連続したｎ文字の文字列がｎ行続く）
$C_{11}C_{12}$ … C_{1n}　← １行目の文字列（連続したｎ文字）
$C_{21}C_{22}$ … C_{2n}　← ２行目の文字列（連続したｎ文字）
⋮
$C_{n1}C_{n2}$ … C_{nn}　← ｎ行目の文字列（連続したｎ文字）

（マップデータの例）

```
5
#####
#....
#####
#....
#....
```

（1）　/home/pub/sample/prog14-01.c は、標準入力より入力したマップデータを2次元配列に格納し、入力したマップをそのままの形で表示するプログラムです。

/home/pub/sample/map.dat は、上の例に示したマップデータのファイルです。これをカレントディレクトリにコピーし、完成したプログラムに入力すると下の実行例ような表示が得られます。

なお、このマップデータをファイルから入力するには、この実行例のように入力リダイレクションを使用します。

（実行例）

```
$ a.out < map.dat ↵
#####
#....
#####
#....
#....
```

このプログラムでは、マップデータの入力は readMap() という関数で、入力したマップの表示は showMap() という関数で行っています。2つの関数の外部仕様は、ソースファイル内のコメントで示してある通りです。**2つの関数を仕様通りに実装**し、プログラムを完成させなさい。

(prog14-01.c)

（2）　/home/pub/sample/prog14-02.c は、読み込んだマップを 90°右回転、90°左回転、左右反転、上下反転したものをそれぞれ順に表示するプログラムです。マップの回転や反転は、transformMap() という関数で行っています。（1）で示したマップデータをこのプログラムに入力すると、右図のように表示されます。

transformMap() 関数の外部仕様は（1）と同様、ソースファイル内にコメントで示してあります。**prog14-02.c に（1）で作った2つの関数 readMap() と showMap() を追加したうえで、transformMap() 関数を仕様通りに実装**しプログラムを完成させなさい。

なお、transformMap() 関数の実装の際、必要なら各自で新たな関数を追加しても構いません。

（prog14-02.c）

レポート

次のものをE-mailで提出しなさい。
・［問題13］，［問題14］のプログラム（添付ファイル）と実行結果（本文）
・演習の感想（本文）

（提 出 先）　E-mail: k-kenta@hamako-ths.ed.jp　件名：プロ技演習13～14
（提出期限）　　／　（　　）　期限厳守

（実行例）

```
$ a.out < map.dat

元の図形
#####
#....
#####
#....
#....

90°右回転
#####
..#.#
..#.#
..#.#
..#.#

90°左回転
#.#..
#.#..
#.#..
#.#..
#####

左右反転
#####
....#
#####
....#
....#

上下反転
#....
#....
#####
#....
#####
```